ATTENDX

AI-Powered Smart Attendance Management System

Project Team Preparation Guide

Quick revision guide for mentor review, viva, internship evaluation, and project presentation. Designed for 15–20 minute revision.

Team Size: 5 Members

1 PROJECT OVERVIEW

What is ATTENDX?

ATTENDX is an AI-powered smart attendance management system that replaces manual roll-call with **facial recognition**, **GPS geofencing**, **WiFi/IP restrictions**, and **session-based access keys** — ensuring only physically present students can mark attendance.

Problem Statement

- **Proxy attendance** — one student marking for another
- **Manual errors** in paper-based roll calls
- **No location verification** — students mark from home
- **No spoof detection** — students use photos to bypass face systems

Key Features

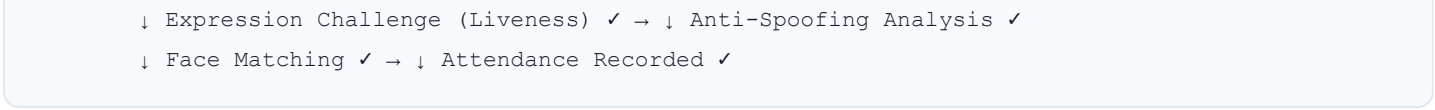
Feature	Description
Face Registration	Students register face biometrics on first login
Face Verification	AI verifies live face before marking attendance
Anti-Spoofing (4 Layers)	Rejects photos, screenshots, screen displays
GPS Geofencing	Only allows attendance within campus boundaries
WiFi/IP Restriction	Validates student's network matches campus
Session Access Keys	Unique key per session distributed by teacher
Start + End Verification	Face verified at both start and end of class
Attendance Analytics	Percentage tracking, trends, shortage alerts
Email Automation	Automatic shortage warning emails
CSV Import	Bulk import students/teachers from CSV

User Roles

Role	Responsibilities
Super Admin	Manages teachers, students, locations, networks, settings
Teacher	Creates sessions, views reports, manages student lists
Student	Registers face, marks attendance, views analytics

System Workflow

```
Admin Setup → Teacher Creates Session → Student Opens App
       ↓ Enter Access Key → ↓ GPS Validation ✓ → ↓ Network Validation ✓
```


↓ Expression Challenge (Liveness) ✓ → ↓ Anti-Spoofing Analysis ✓
↓ Face Matching ✓ → ↓ Attendance Recorded ✓

2 FRONTEND DEVELOPER GUIDE

Tech Stack

React 18 (UI) · **Vite** (build) · **React Router v6** (routing) · **Axios** (API) · **face-api.js** (face detection) · **Framer Motion** (animations) · **Recharts** (charts)

Files & Modules Handled

Authentication

`LoginPage.jsx`	Login + face verification UI
`AuthContext.jsx`	Global auth state
`ProtectedRoute.jsx`	Route guards

Teacher Pages

`TeacherDashboard.jsx`	Session overview
`TeacherAttendanceSession.jsx`	Live sessions
`TeacherReports.jsx`	Reports & exports

Super Admin Pages

`SuperAdminDashboard.jsx`	Overview stats
`TeacherManagement.jsx`	CRUD + CSV import
`StudentManagement.jsx`	CRUD + CSV import
`LocationManagement.jsx`	GPS boundaries
`NetworkManagement.jsx`	IP/WiFi rules

Student Pages

`StudentAttendance.jsx`	Mark attendance flow
`AttendanceAnalytics.jsx`	Charts & trends
`StudentProfile.jsx`	Profile management

Routing Structure

```
/login → LoginPage | /superadmin → Admin Dashboard (role: super_admin)
/teacher → Teacher Dashboard (role: teacher) | /student → Student Dashboard (role: student)
/student/mark → StudentAttendance | /* → NotFoundPage (404)
```

Mentor Q&A

Q: What did you do in frontend?

A: I built the complete UI using React 18 with Vite. I created role-based dashboards for admin, teacher, and student. The most complex component is StudentAttendance.jsx which handles GPS validation, camera access, expression-based liveness detection, and face matching — all in a step-by-step wizard flow.

Q: How is routing handled?

A: We use React Router v6 with a ProtectedRoute wrapper. If a user isn't authenticated, they're redirected to /login. Each route checks the user's role and redirects to the correct dashboard. For GitHub Pages deployment, we use a dynamic basename and a 404.html fallback for SPA routing.

Q: How does attendance data appear on screen?

A: StudentAttendance.jsx fetches active sessions from the API. When a student selects a session, they go through a 4-step wizard: Access Key → GPS/Network → Expression Challenge → Face Scan. The backend returns the result and the UI shows success/failure with accuracy percentage.

3 BACKEND DEVELOPER GUIDE

Tech Stack

Flask (framework) · **Flask-JWT-Extended** (auth) · **SQLAlchemy** (ORM) · **bcrypt** (passwords) · **face_recognition/dlib** (AI) · **OpenCV** (image processing) · **APScheduler** (cron jobs)

API Routes

Auth Routes (/api/auth/)

Endpoint	Purpose
POST /login	Email + password login
POST /enroll-face	Face registration
POST /verify-login-face	Face 2FA
POST /refresh	Token refresh
POST /logout	Session end
GET /me	Current user

Attendance Routes (/api/attendance/)

Endpoint	Purpose
POST /start	Start session
POST /end	End session
POST /verify-access-key	Validate key
POST /validate	GPS + network check
POST /mark	Mark attendance
GET /active-sessions	Student's sessions

Request Flow

```
Frontend (React) → Axios POST /api/attendance/mark
    → Flask Route Handler → Validate Access Key ✓
    → Validate GPS ✓ → Validate Network ✓
    → Anti-Spoofing Pipeline ✓ → Face Recognition ✓
    → Save AttendanceRecord to DB → JSON Response → Frontend
```

Mentor Q&A

Q: How does login work?

A: Login is multi-step. Step 1: Email + password validated with bcrypt. For students, a face_challenge_token is returned. Step 2: Student passes expression liveness challenge and submits face frames. Backend runs anti-spoofing analysis, matches face against stored encodings. Only if all checks pass are JWT tokens issued.

Q: How is attendance stored?

A: Each record is a row in AttendanceRecord with student_id, session_id, start_time, end_time, start_confidence, end_confidence, gps_validated, network_validated, and status (present/absent/partial).

Q: What happens after attendance submission?

A: The /mark endpoint validates access key → checks GPS → validates IP → runs 4-layer anti-spoofing → performs face recognition. If all pass, it creates/updates an AttendanceRecord with the confidence score, GPS, and network status.

4 DATABASE DEVELOPER GUIDE

Database: SQLite (dev) / PostgreSQL (production-ready via SQLAlchemy ORM)

Tables & Key Fields

users

Column	Type
id	Integer PK
email	String UNIQUE
password_hash	String (bcrypt)
name	String
role	super_admin/teacher/student

students

Column	Type
id	Integer PK
user_id	FK → users.id
roll_number	String UNIQUE
college_name	String
face_registered	Boolean

face_encodings

Column	Type
id	Integer PK
student_id	FK → students.id
encoding_data	LargeBinary (128-dim)

attendance_sessions

Column	Type
id	Integer PK
teacher_id	FK → users.id
subject	String
access_key	String
start_time / end_time	DateTime
status	active/completed

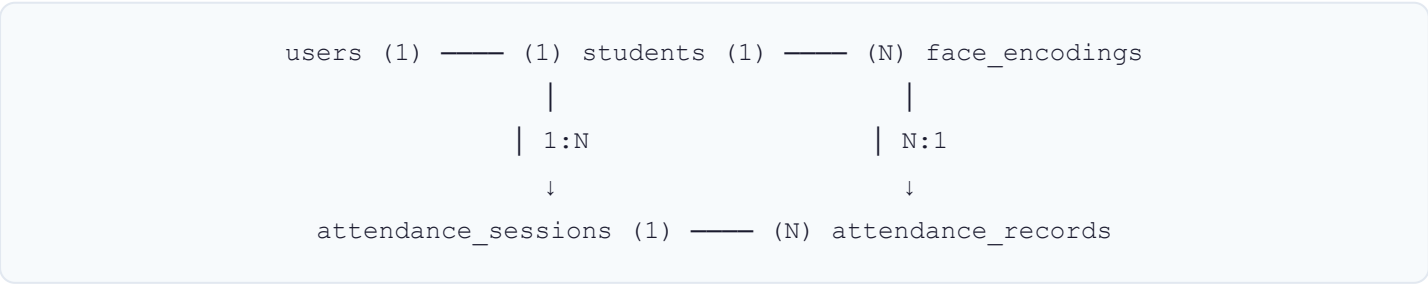
attendance_records

Column	Type
id	Integer PK
student_id	FK → students.id
session_id	FK → sessions.id
start_time / end_time	DateTime
start_confidence	Float
status	present/partial/absent

allowed_locations / allowed_networks

Column	Type
name, lat, lng, radius	GPS geofences
name, ip_address	Network rules

ER Diagram



Mentor Q&A

Q: Why separate users and students tables?

A: The users table handles authentication for ALL roles. The students table holds student-specific data (roll number, face status). This normalized design avoids NULL columns for non-student users and allows role-specific extensions.

Q: How are attendance records linked?

A: Each attendance_record has two foreign keys: student_id → students and session_id → attendance_sessions. This creates a many-to-many relationship. We can query "all students in a session" or "all sessions for a student" efficiently.

5 AI MODEL DEVELOPER GUIDE

Models Used

Model	Library	Purpose
TinyFaceDetector	face-api.js (frontend)	Real-time face detection in browser
FaceExpressionNet	face-api.js (frontend)	Expression classification for liveness
dlib HOG	face_recognition (backend)	Face detection in submitted images
dlib ResNet-34	face_recognition (backend)	128-dim face embedding generation
OpenCV Heuristics	OpenCV (backend)	Texture/frequency anti-spoofing

How Face Recognition Works

Registration: Camera captures 20-30 images → dlib detects face → generates 128-dim embedding per image → stored in face_encodings table

Verification: Camera captures live frame → dlib generates embedding → Euclidean distance vs stored embeddings → if distance < 0.55 → MATCH ✓

Anti-Spoofing: 4 Independent Layers

Layer 1: Random Expression Challenge (Frontend)

Pool of 5 expressions (Neutral, Happy, Surprised, Angry, Sad). Pick 3 randomly. User must perform each. A photo can't change expression → **FAILS**.

Layer 2: Texture Analysis (Backend)

`cv2.Laplacian(face)` → variance. Real skin > 15 (rich texture). Screen display < 15 (flat, resampled) → **FAILS**.

Layer 3: Frequency Analysis (Backend)

`np.fft.fft2(face)` → frequency spectrum. Screen pixels create moiré → high-freq ratio > 0.35 → **FAILS**.

Layer 4: Temporal Motion (Backend)

Compare face across frames. Real face has micro-movements → motion > 1.8. Photo is static → motion < 1.8 → **FAILS**.

Mentor Q&A

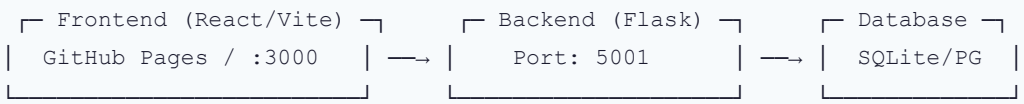
> **Q: How does face recognition work?**
>
> *A: We use dlib's ResNet-34 to generate 128-dimensional face embeddings. During registration, we store 20+ embeddings. During verification, we compute Euclidean distance between the live embedding and stored ones. Distance below 0.55 = match. We also check against ALL students to prevent impostor attacks.*

Q: How do you prevent fake attendance?

A: Four layers: (1) Random expression challenge — photo can't change expressions. (2) Texture analysis — Laplacian variance differs for screens vs real skin. (3) Frequency analysis — FFT detects screen moiré patterns. (4) Temporal analysis — real faces have micro-movements between frames. At least 2 of 4 backend checks must pass.

6 INTEGRATION & TESTING GUIDE

System Architecture



Deployment

Component	Development	Production
Frontend	localhost:3000	manaspediredla.github.io/attendx
Backend	localhost:5001	Cloud (Render/Railway)
Base URL	/	/attendx/

Testing Performed

Test Area	What Was Tested	Result
Login	Email/password, JWT tokens, role redirection	✅ Pass
Face Registration	20+ images capture, encoding storage	✅ Pass
Face Verification	Match accuracy, impostor rejection	✅ Pass
Anti-Spoofing	Phone photo, screenshot, printed image	✅ Rejected
GPS Validation	Campus boundary, radius check	✅ Pass
Network Validation	IP matching, public IP check	✅ Pass
Access Key	Correct → pass, wrong → reject	✅ Pass
Session Lifecycle	Start → Mark → End → Complete	✅ Pass
CSV Import	Bulk student/teacher upload	✅ Pass
Reports	Attendance percentages, date filtering	✅ Pass

Mentor Q&A

> **Q: What testing was performed?**
>
> *A: End-to-end manual testing across all modules. For auth: login flows for all 3 roles. For attendance: full flow (session → key → GPS → network → liveness → face → record). For anti-spoofing: real faces, phone photos, screenshots. Also tested edge cases like expired sessions, duplicate attendance, and network timeouts.*

Q: How did you validate the project?

A: Multiple levels: (1) Component testing — each page individually. (2) API testing — each endpoint. (3) Integration testing — full frontend-to-database flow. (4) Security testing — proxy attempts with photos. (5) User acceptance — students tested in a classroom setting.

7 FEATURE EXPLANATION GUIDE

🔐 Face Attendance

Purpose: Replace manual roll-call with facial recognition

How: Student faces camera → Expression challenge → Backend matches 128-dim embedding → Attendance recorded

Modules: StudentAttendance.jsx · face_service.py · attendance.py

📍 GPS Restriction

Purpose: Ensure student is physically on campus

How: Browser Geolocation gets lat/lng → Backend calculates Haversine distance to campus → Reject if > radius

Modules: validation_service.py · allowed_location.py · LocationManagement.jsx

🔑 Session Access Key

Purpose: Prevent attendance in wrong session

How: Teacher sets unique key → Student enters key → Backend validates match

Modules: TeacherAttendanceSession.jsx · attendance_session.py

👨‍🏫 Teacher Management

Purpose: Admin manages teacher accounts

How: Admin creates teacher via form or CSV → Teacher logs in → Creates sessions

Modules: TeacherManagement.jsx · admin.py · csv_service.py

📊 CSV Import

Purpose: Bulk import students/teachers from spreadsheet

How: Admin uploads CSV → Backend parses, validates → Creates accounts + profiles

Modules: csv_service.py · teacher_csv_service.py · StudentManagement.jsx

📈 Attendance Reports

Purpose: Track attendance percentages and trends

How: Query records grouped by student/session → Calculate % → Display charts

Modules: TeacherReports.jsx · report_service.py · analytics.py

📧 Shortage Email Alerts

Purpose: Alert students with low attendance

How: APScheduler checks % → If < threshold → Sends warning via SMTP

Modules: email_service.py · scheduler.py

🛡️ Anti-Spoofing System

Purpose: Reject photos, screenshots, screen displays

How: 4 layers — expression challenge + texture + frequency + temporal motion

Modules: antispoof_service.py · StudentAttendance.jsx · LoginPage.jsx

⏱️ Start + End Verification

Purpose: Verify student was present for full class

How: Face verified at start AND end → Both timestamps stored → Status: present/partial/absent

Modules: attendance_service.py · StudentAttendance.jsx

8 PROJECT FLOW & QUICK REVISION

Complete Project Workflow

```
ADMIN SETUP
  ├── Create Teacher Accounts (manual or CSV)
  ├── Create Student Accounts (manual or CSV)
  ├── Configure Campus GPS Locations
  └── Configure Allowed Networks
          ↓
TEACHER CREATES SESSION
  ├── Set Subject, Date, Time Window
  ├── Generate Access Key
  └── Activate Session
          ↓
STUDENT JOINS SESSION
  ├── Step 1: Enter Access Key ✓
  ├── Step 2: GPS + Network Validation ✓
  ├── Step 3: Expression Liveness Challenge ✓
  │      └── Neutral → Random Expression 1 → Random Expression 2
  ├── Step 4: Backend Anti-Spoofing ✓
  │      └── Texture + Frequency + Color + Motion
  ├── Step 5: Face Recognition Match ✓
  └── ATTENDANCE MARKED (Start)
          ↓
END SESSION → Same liveness + face check → Both times recorded
          ↓
REPORTS → Teacher views reports → Student sees % → Email alerts sent
```

🚀 Things Every Team Member MUST Know

Project Basics

- ✅ ATTENDX = AI-powered attendance system with 3 roles: Admin, Teacher, Student
- ✅ Tech Stack: React (frontend) + Flask (backend) + SQLite (database)
- ✅ Face recognition model: **dlib ResNet-34** (128-dimensional embeddings)
- ✅ Liveness detection: **FaceExpressionNet** (browser-based, 7 expressions)

How Attendance Works

- ✅ Access Key → GPS → Network → Liveness → Anti-Spoof → Face Match → Recorded
- ✅ Both START and END verification required
- ✅ Status: Present (both) / Partial (one) / Absent (none)

Anti-Spoofing (4 Layers)

- ✅ Layer 1: Random expression challenge (photo can't change expression)

- ✅ Layer 2: Laplacian texture analysis (screens have flat texture)
- ✅ Layer 3: FFT frequency analysis (detects screen moiré patterns)
- ✅ Layer 4: Temporal motion (real face has micro-movements)

Key Numbers to Remember

- ✅ **128** dimensions per face embedding
- ✅ **0.55** = face match distance threshold
- ✅ **20** minimum face images for registration
- ✅ **5** expression confirmations per challenge step
- ✅ **3** random expressions from pool of **5**

Stay confident. Explain YOUR role clearly. Connect your work to the system. Good luck! 🎓